

Smart Campus Mobility

Real-Time Campus Mobility & Ride Management Platform — Design Document

By Veldandi Ruthwik – 23115158, Chinni Krishna – 23115066 , Nallapati Sanjana - 23115089

1 • Problem Understanding

The IIT Roorkee campus spans a large geographical area and depends on e-rickshaws for last-mile transport, but ride requests and driver availability are coordinated through fragmented, informal channels. The result is the classic dispatch failure mode: idle drivers in one zone while passengers wait in another, no shared view of who is free, and no record of demand to plan around. The task is to replace that with a single real-time platform that connects **passengers**, **drivers**, and an **operations team** through one coordinated digital system.

What each actor needs

- **Passenger** — register, see who is nearby, request a ride between named campus zones (or schedule one), watch live status as a driver is assigned and approaches, pay, and rate.
- **Driver** — onboard with vehicle & license details, get verified, go online/offline, see incoming requests on a live board, accept/reject, run the trip through an OTP-verified start, and track earnings & ratings.
- **Admin / Ops** — approve drivers, watch every ride in flight, and read demand analytics & forecasts to position the fleet.

The hard engineering problems

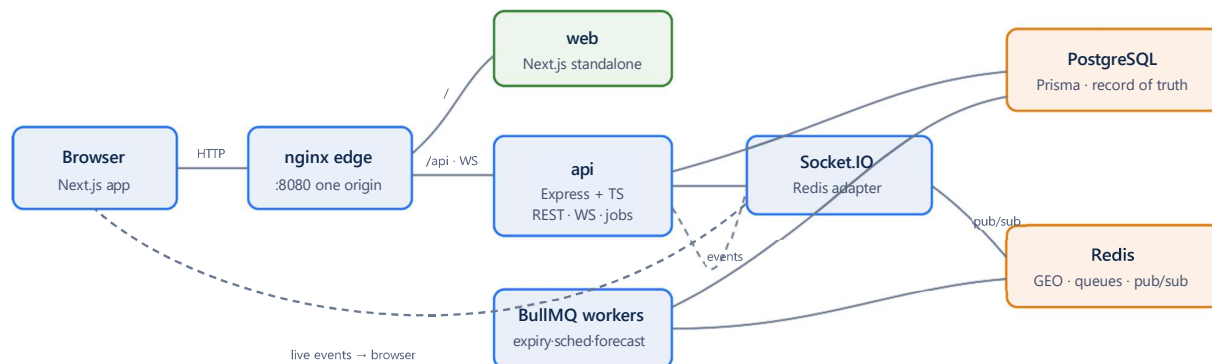
- **Concurrency correctness** — a ride must be assignable to exactly one driver, even when two tap *Accept* in the same millisecond.
- **Real-time first** — every state change is *pushed*, never polled: live ride status, driver presence, and assignment notifications.
- **State consistency** — ride state, assignment, and availability stay synchronized across passenger, driver, and admin views at all times.
- **Geospatial handling** — "drivers near me" answered in real time.
- **Scalability** — the topology must grow past a single process.

Scope & design priorities

Per the brief, we prioritize a **reliable core ride-management workflow** over a wide-but-shallow feature list. The design optimizes, in order, for: (1) correctness under concurrency, (2) real-time delivery of every state change, and (3) demo reproducibility — one `docker compose up` yields a fully seeded, explorable system. All mandatory features (auth & profiles, driver availability, ride request workflow, real-time updates, full ride lifecycle, driver dashboard, ratings) are implemented, plus the optional demand analytics & forecasting and simulated digital payments.

2 • System Architecture

A single nginx origin fronts a Next.js web app and an Express API; the API process hosts REST, the Socket.IO server, and BullMQ background workers, backed by PostgreSQL (system of record) and Redis (presence, geo-index, queues, socket fan-out).



Topology — all browser traffic enters through one nginx origin (cookies stay first-party, no CORS on the happy path).

Why a single API process

At campus scale, hosting REST + Socket.IO + workers in one Express process is the right shape. The seams to split it later are already in place: sockets ride the Redis adapter, and workers communicate *only* through the queue and database — so adding API replicas changes nothing about delivery or job semantics.

Layered backend

Each domain is a module — `auth · rides · drivers · ratings · zones · users · admin` — with `router → controller → service` layers. Services never import the socket layer; they emit typed **domain events** on an in-process bus that the socket and queue modules subscribe to. Business logic is therefore unit-testable without a running socket server.

3 · Database Schema

PostgreSQL 15 via Prisma ORM. Nine models; identifiers are collision-resistant `cuid`s. Indexes are placed on the hot paths: the dispatch board, "my rides", driver history, and demand analytics.

Model / table	Purpose & key fields	Notable indexes & constraints
User	One row per human. <code>email</code> · <code>phone</code> unique, <code>passwordHash</code> (bcrypt), <code>role</code> ∈ PASSENGER/DRIVER/ADMIN.	<code>@@index(role)</code> ; email & phone unique.
DriverProfile	Vehicle-bearing identity that <i>serves</i> rides. License, vehicle, <code>verificationStatus</code> , <code>status</code> (OFFLINE/ONLINE/BUSY), live <code>currentLat/Lng</code> , denormalized <code>ratingAvg</code> / <code>ratingCount</code> / <code>totalRides</code> .	<code>@@index(status, verificationStatus)</code> for "online & approved" dispatch query. 1:1 to User.
Ride	Central aggregate. Pickup/drop labels + coords, <code>status</code> (7-state), fares, <code>startOtp</code> , payment fields, full timestamp trail.	4 composite indexes incl. (<code>status, requestedAt</code>) board & (<code>scheduledFor, code</code>) unique.
RideEvent	Append-only audit log of every transition (REQUESTED...EXPIRED). Powers the admin live feed and makes rides replayable.	<code>@@index(rideId, createdAt)</code> .
Rating	One per completed ride. Stars 1–5 + optional comment; written in the same txn that rolls up the profile average.	<code>rideId</code> unique (one rating/ride); <code>@@index(driverId, createdAt)</code> .
DriverDocument	Verification uploads (LICENSE / VEHICLE_RC / ID_PROOF) — server-generated file keys.	FK → DriverProfile (cascade).
CampusZone	Named pickup/drop locations (gates, bhawans, departments) with lat/lng.	<code>name</code> unique.
RefreshToken	SHA-256 <i>hash</i> of each opaque refresh token + expiry/revocation, enabling rotation & theft detection.	<code>tokenHash</code> unique; <code>@@index(userId)</code> .
DemandForecast	Predicted rides per zone per hour; <code>modelVersion</code> seams in a better model later without a schema change.	<code>@@unique(zoneName, forHour)</code> .

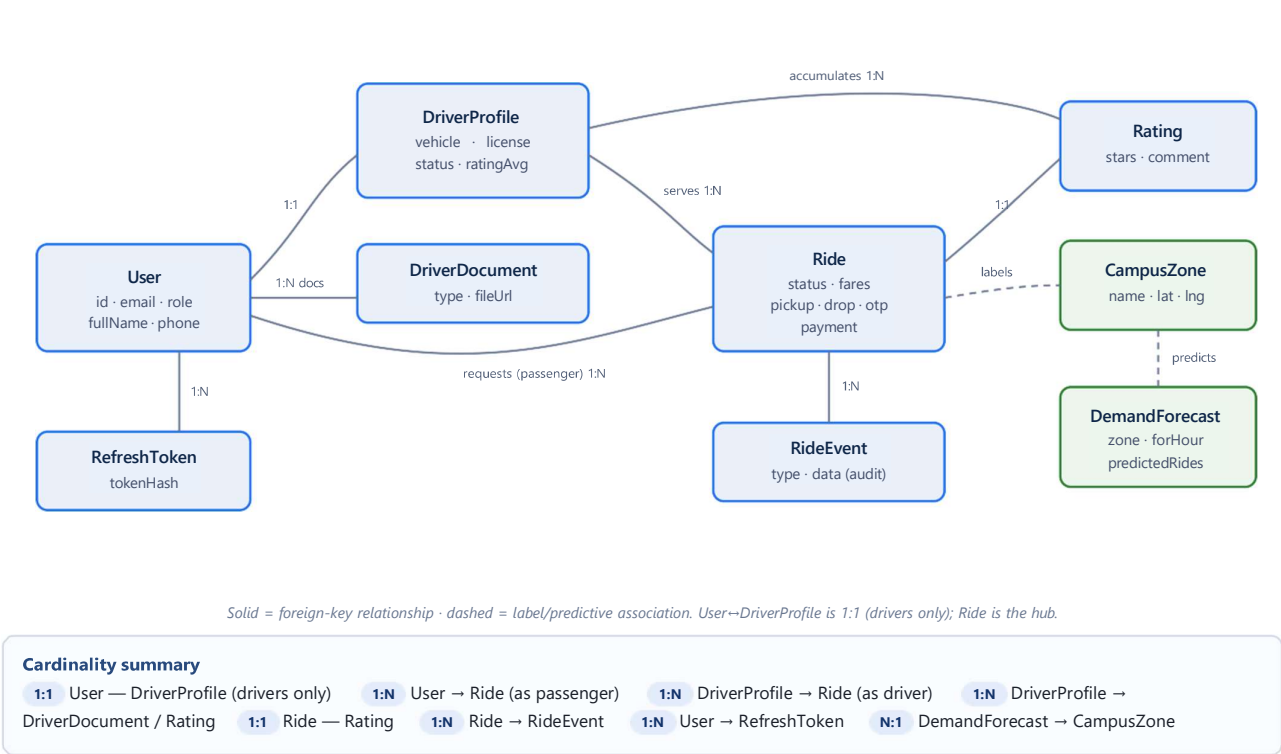
Design decision: rides reference the profile, not the user

Ride. `driverId` → DriverProfile. `id` , not User. `id` . Rides are served by a vehicle-bearing profile, and every aggregate a driver cares about — earnings, rating roll-ups, totals — hangs off the profile. The serializer always nests `driver.user.fullName/phone` so clients never need a second lookup.

Design decision: denormalized reputation

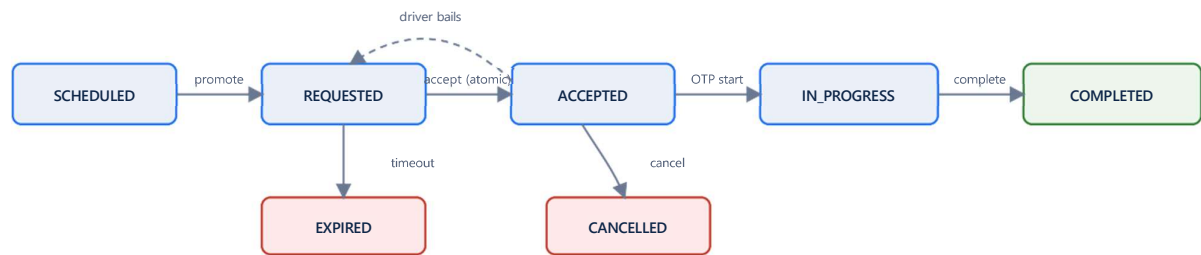
`ratingAvg` & `ratingCount` live on the profile and are updated inside the same transaction that inserts the `Rating` row. The dispatch board can sort and display driver reputation without running an aggregate query on every render.

4 · Entity Relationship Diagram



5 · Ride Lifecycle & Concurrency Model

The 7-state machine is the heart of the system. It is enforced *twice*: once in application code for readable errors, and once **structurally in the database write**.



States: *SCHEDULED → REQUESTED → ACCEPTED → IN_PROGRESS → COMPLETED*, with *EXPIRED & CANCELLED* as terminal off-ramps.

Guarded conditional writes

Every status change is a guarded `updateMany` whose `WHERE` includes the expected current status:

```
await tx.ride.updateMany({
  where: { id, status: from }, // the guard
  data: { status: to, _ },
});
if (count = 0) throw ApiError(409); // lost the race
```

A concurrent transition makes the second writer match **zero rows** rather than silently overwriting. This is optimistic concurrency without version columns — *the status itself is the version*.

The accept race, resolved

Two drivers tapping *Accept* within milliseconds is the sharpest edge. The first `updateMany` matches one row (count = 1) and assigns the driver; the second matches zero (count = 0) and gets a clean `409 RIDE_TAKEN`. A broadcast `ride: unavailable` then clears the card from every *other* online driver's board.

No locks, no queues in the hot path — one indexed conditional write decides the winner, which is exactly the "a ride can only be assigned to a single driver" guarantee the brief requires.

6 · Real-Time Communication

This is the core engineering component. Services emit typed domain events on an in-process bus (`ride.requested` , `ride.updated` , `driver.presence` , `driver.location`); the socket module translates them into room broadcasts. The broadcast topology lives in exactly one file.

Room	Members	Receives
<code>user:{userId}</code>	that user's devices	<code>ride: update</code> for their own rides
<code>drivers:online</code>	drivers toggled online	<code>ride: requested</code> , <code>ride: unavailable</code>
<code>ride:{rideId}</code>	participants who opted in	<code>driver: location</code> , <code>ride: update</code>
<code>admins</code>	operations staff	everything above + <code>driver: presence</code>

Presence & geo

Driver GPS pings (throttled client-side to ~4s) update a **Redis GEO index** for nearby-driver lookups and persist to Postgres at a lower rate — Redis is the record of *now*, Postgres of *roughly where*. A driver disconnect starts a **45-second grace timer** before flipping ONLINE→OFFLINE, so a Wi-Fi blip doesn't dequeue a working driver.

Scale-ready from day one

Socket.IO runs on the `@socket.io/redis-adapter` , so adding API replicas changes nothing about message delivery. The handshake authenticates with the same short-lived JWT as REST, and room joins derive from the *verified* identity — never from client claims.

7 · API Overview

REST base `/api/v1`. Bearer access token (15 min) + httpOnly refresh cookie. Success payloads are wrapped in a named key; errors share one envelope `{ error: { code, message } }`. Selected endpoints:

Auth & profile

POST <code>/auth/register</code>	passenger signup
POST <code>/auth/register/driver</code>	driver signup (PENDING)
POST <code>/auth/login</code>	issue token pair
POST <code>/auth/refresh</code>	rotate refresh + new access
POST <code>/auth/logout</code>	revoke refresh
GET <code>/auth/me</code>	session user (+profile)
PATCH <code>/users/me</code>	update name/phone
GET <code>/zones</code>	seeded campus locations

Ride lifecycle

POST <code>/rides</code>	request (or schedule)
GET <code>/rides/active</code>	current non-terminal ride
GET <code>/rides/open</code>	driver dispatch board
POST <code>/rides/:id/accept</code>	atomic assign · 409 if taken
POST <code>/rides/:id/start</code>	OTP-gated → IN_PROGRESS
POST <code>/rides/:id/complete</code>	final fare + settle payment
POST <code>/rides/:id/cancel</code>	cancel / driver bail
POST <code>/rides/:id/rating</code>	1–5 stars, one per ride

Driver availability & dashboard

PATCH <code>/drivers/me/availability</code>	ONLINE / OFFLINE
POST <code>/drivers/me/documents</code>	upload verification doc
GET <code>/drivers/nearby</code>	Redis GEO radius query
GET <code>/drivers/me/dashboard</code>	stats · 14-day daily · ratings

Admin / operations

GET <code>/admin/overview</code>	live KPI tiles
GET <code>/admin/rides/live</code>	every ride in flight
POST <code>/admin/drivers/:id/verification</code>	approve / reject
GET <code>/admin/analytics/demand</code>	by-hour · weekday · hotspots
GET <code>/admin/analytics/forecast</code>	next-hours per-zone forecast

Error envelope & representative codes

`400` VALIDATION_ERROR · BAD_OTP `401` TOKEN_INVALID · BAD_CREDENTIALS · REFRESH_INVALID `403` FORBIDDEN · NOT_VERIFIED · NOT_YOUR_RIDE `409` RIDE_TAKEN · ILLEGAL_TRANSITION · RIDE_ALREADY_ACTIVE `429` rate-limited

Every `ride` response is fully serialized (passenger, nested driver profile + user, fares, `startOtp`) so clients replace local state wholesale rather than patching — identical shape over REST and sockets.

8 · Background Jobs & Analytics

BullMQ queues (on the same Redis)

- ride-expiry** — per request; moves still- `REQUESTED` rides to `EXPIRED` at the dispatch timeout. A late job against an accepted ride is a harmless no-op (the guarded write protects it).
- scheduled-rides** — one delayed job per future booking; promotes `SCHEDULED` → `REQUESTED` with a configurable lead, then normal dispatch takes over.
- forecasts** — repeatable hourly scheduler recomputing demand predictions.

Job IDs are deterministic (`expire: {rideId}: {ts}`) so retries and re-enqueues stay idempotent.

Demand analytics & forecasting

Campus demand is dominated by two seasonalities — hour-of-day and day-of-week. The forecaster is deliberately **seasonal-naive with exponential recency weighting**: per (`zone`, `hourOfDay`, `dayOfWeek`) bucket it takes the weighted mean of the last four matching weeks, halving weight per week of age, writing per-zone-per-hour predictions for the next 24h.

It trains in a single SQL pass, is fully explainable to a transport office, and beats anything needing hyperparameters at this data volume. The admin *Demand* tab adds descriptive analytics (hour histogram, weekday profile, pickup hotspots).

9 · Design Decisions

① One nginx origin, one API process

All browser traffic enters at `:8080`, keeping cookies first-party and removing CORS from the happy path. REST, Socket.IO, and workers share one process — correct at campus scale, and pre-seamed for splitting (Redis socket adapter + queue/DB-only worker coupling). **Trade-off:** a single deploy unit today; horizontal scale is a config change, not a rewrite.

② Domain events over direct coupling

Services emit typed events on an in-process bus instead of importing the socket layer. This makes ride logic unit-testable without a socket server, keeps broadcast topology in one file, and lets us swap to an external broker later by changing the bus implementation — not business code. **Trade-off:** one indirection hop for a large gain in testability and modularity.

③ Status-as-version optimistic concurrency

Rather than row locks or a version column, the expected status is the guard in every write. This is lock-free, index-backed, and makes the single-driver-assignment invariant a property of the database, not of careful application code. **Trade-off:** the loser of a race gets a `409` to handle gracefully (the UI already does) instead of blocking.

④ Security: short access token + rotating hashed refresh

15-min in-memory access JWT (never `localStorage`) + 48-byte opaque refresh token stored only as a SHA-256 hash in an `httpOnly`, path-scoped cookie. Every refresh rotates and revokes the old token, so a replayed token is both rejected and a theft signal. Passwords are `bcrypt` (cost 10); auth routes are rate-limited; Zod validates every input; `helmet` + CORS allow-list + 1 MB body cap harden transport. **Trade-off:** a refresh round-trip on token expiry, mitigated by a single-flight axios lock.

⑤ OTP-verified trip start

The passenger's screen shows a 4-digit code the driver must type to move `ACCEPTED` → `IN_PROGRESS`. This proves physical co-presence before fare state advances and blocks a driver from manipulating ride state alone. The enforcement point is the transition check, with the UI split providing the human step.

⑥ Postgres + Redis split of responsibility

Postgres is the durable system of record (rides, users, ratings, audit trail); Redis owns the *now* — GEO index for nearby drivers, BullMQ queues, and Socket.IO pub/sub fan-out. Live location is high-frequency and ephemeral, so it lives in Redis and is persisted to Postgres only at a throttled rate. **Trade-off:** two stores to run, justified by clean separation of hot vs. durable data.

⑦ Deterministic fares & simulated payments

A flat ₹10 fare between any two zones keeps the demo free of pricing edge cases; UPI rides flip to `PAID` on completion and cash rides record collection. The `paymentStatus` field and completion hook are exactly where a real PSP webhook would land — a config change, not a redesign.

⑧ Frontend: two Zustand stores, contract mirrored from the API

`auth` owns the session (in-memory token + silent refresh); `live` owns everything the socket pushes, binding listeners once per connection. Server payload shapes live in one `types.ts` written from the API serializers, so contract-drift bugs are structurally hard to reintroduce — the whole app typechecks against those shapes in CI.

Reproducibility & honest scope

`prisma db push` at container boot trades migration history for a zero-friction first run (the right call for evaluation; `DEPLOYMENT.md` documents the switch to `migrate deploy` for production). Live maps render distance-to-pickup rather than tiles to keep the demo API-key-free — the location event stream already carries everything a Leaflet/MapLibre layer would need. Growth directions (push notifications, surge fares fed by the forecaster, multi-stop shuttle routing over the same zone graph) all sit on seams the current design intentionally left exposed.